

Git Commands & Railway Deployment

Lecture 3 — Version Control Reference & The Full Deploy Story

Every Git command explained with real-world analogies, followed by the complete, unfiltered account of deploying this project to Railway — including every real error hit along the way and exactly why each fix worked.

Companion to: Lecture 2 — PostgreSQL, Docker & Alembic Setup Guide

Part 1 — Git Commands, Explained to Stick

Git is a version control system — it records snapshots of your project over time, so nothing is ever truly lost and every change can be traced, compared, or undone. Think of it like a save-game system for your codebase: every commit is a checkpoint you can always return to.

1.1 The three-zone mental model

Almost every Git command moves a change between three zones. Understanding these three zones makes every command below make sense immediately, instead of feeling like memorized magic words.

Zone	What it means
Working Directory	Your actual files on disk, exactly as you're editing them right now.
Staging Area	A holding area — files you've marked as "ready to be included in the next commit," but haven't committed yet.
Repository (.git)	The permanent, saved history — every commit ever made, stored locally in the hidden .git folder.

Real-world analogy

Working Directory is your kitchen counter while cooking — messy, in progress. Staging Area is the tray you've plated and are about to serve — chosen and ready. Repository is the recipe book — once a dish is written in, it's permanent history you can always look back on.

1.2 Core commands — one at a time

git init

```
git init
```

Turns a plain folder into a Git repository by creating a hidden .git folder. This is a one-time setup per project — run once, at the very start.

Real-world example

You just finished building the CLI version of the task manager in a folder with no version control. Running git init there is the moment that folder starts being tracked — like deciding "from this point on, I'm keeping a diary of every change I make here."

git status

```
git status
```

Shows the current state of all three zones at once: which files are modified but not staged, which are staged and ready to commit, and which are untracked entirely. This is the single most-used command in Git — run it constantly, before and after almost every other command, to confirm what's actually about to happen.

git add

```
git add .          # stage everything in this folder
git add database.py # stage one specific file
```

Moves changes from the Working Directory into the Staging Area — marking them as "include this in the next commit." Using `.` stages every changed/new file at once; naming a specific file stages only that one.

Real-world example

You changed `database.py` to read from an environment variable, but you also have three personal notes files sitting in the same folder that you don't want tracked. Instead of `git add .`, you'd run `git add database.py` — deliberately choosing exactly what goes into the next snapshot, the way you'd pack only the specific items you need for a trip rather than throwing your whole room into a suitcase.

git commit

```
git commit -m "Read DATABASE_URL from environment for Railway deployment"
```

Takes everything currently staged and permanently saves it as a new snapshot in the repository's history, along with a message describing what changed and why. Every commit also automatically records who made it and when.

Real-world example

This is exactly the commit from the deployment story — the moment `database.py` was fixed to read from Railway's environment variable instead of a hardcoded local address. The message itself becomes a permanent, searchable log entry — six months from now, `git log` would still show exactly when and why that change was made.

git log

```
git log --oneline -5      # last 5 commits, one line each
```

Shows the commit history — a chronological record of every snapshot ever taken. `--oneline` compresses each commit to a single line (short hash + message) instead of the full detailed view, and `-5` limits it to the last 5 entries.

git config

```
git config --global user.name "Anssh"
git config --global user.email "your-email@example.com"
```

One-time identity setup — tells Git who is making commits on this machine. Required before your very first commit; `--global` means it applies to every repository on this computer, not just the current project.

git rm --cached

```
git rm --cached "Step 1.pdf" "Step 2.docx"
```

Removes files from Git's tracking without deleting them from disk. The files stay exactly where they are on your computer — only their presence in future commits is removed.

Real-world example

This is exactly what fixed the repo cleanup — `Step 1.pdf` and `Step 2.docx` had been accidentally committed

alongside the actual project code. `git rm --cached` removed them from the repository's tracking while leaving the actual PDF/Word files untouched on the Desktop — like removing a photo from a shared album without deleting it from your camera roll.

git mv

```
git mv main.py cli_version.py
```

Renames or moves a file, and stages that rename in one single step (equivalent to a manual rename plus `git add`). Git tracks this as a rename rather than a delete-plus-new-file, preserving the file's history.

Real-world example

Railway kept defaulting to running `main.py`, which was actually the old CLI version expecting keyboard input — causing a crash the moment it ran on a server with no terminal attached. `git mv main.py cli_version.py` renamed it out of the way in one clean, tracked step, so the FastAPI app in `api.py` became the unambiguous entry point.

.gitignore (not a command — a file)

```
__pycache__/  
*.pyc  
.env  
venv/  
*.db
```

A plain text file listing patterns of files/folders Git should never track — even if someone runs `git add .`, anything matching these patterns is silently skipped. Critical for excluding secrets (`.env`), machine-specific junk (`__pycache__`), and anything that would bloat or compromise the repo.

git remote add

```
git remote add origin https://github.com/syedansfraz/fastapi-task-manager.git
```

Links your local repository to a remote address — a copy of the project living somewhere else (in this case, GitHub). `origin` is just a conventional nickname for that remote URL, used so future commands don't need the full URL retyped.

git branch -M

```
git branch -M main
```

Renames the current branch to `main`. GitHub's default branch convention is `main`; some older Git installs default to `master`, so this standardizes the name.

git push

```
git push -u origin main    # first push: also sets the default target  
git push                  # every push after that
```

Uploads local commits to the remote repository (GitHub). The `-u` flag (only needed the first time) links your local `main` branch to `origin's main` branch as the default, so every push after that can just be `git push` with no extra arguments.

Real-world example

Every single fix during the Railway deployment — the database.py env-var fix, the Procfile removal, the driver rewrite — became real only once pushed. Railway watches the GitHub repo, not your local machine; git push is the literal bridge between "I fixed it on my computer" and "the live server actually has the fix."

git pull

```
git pull
```

The reverse of push — downloads and merges changes from the remote repository into your local one. Essential when working across multiple machines, or with teammates who've pushed their own changes.

1.3 The full everyday flow, start to finish

```
git status          # 1. see what changed
git add .           # 2. stage the changes
git commit -m "message" # 3. save a snapshot locally
git push            # 4. upload to GitHub (and trigger Railway's
redeploy)
```

Memory hook

status to look, add to choose, commit to save, push to share. Every single Git session in this project followed exactly this four-step rhythm — the entire deployment debugging loop was really just this cycle, repeated with a different fix each time.

Part 2 — Why Railway, and Not AWS or Another Option

This decision was made deliberately, not by default — worth restating clearly since it shapes everything in Part 3.

2.1 The core reasoning

- Railway lets you deploy directly from a GitHub repo or a Docker image, with a managed Postgres instance provisioned in the same project — meaning almost everything already learned (Docker, Postgres, Alembic) carries over directly, with minimal new platform-specific concepts.
- It is widely considered the simpler, faster-onboarding option for beginners, compared to configuring infrastructure manually.
- Usage-based pricing that scales to zero suits a learning project with no real traffic yet.

2.2 Why not AWS, specifically

AWS is raw cloud infrastructure — hundreds of available services, full control over every layer, and full responsibility for configuring, securing, and maintaining all of it. That's not a weakness of AWS; it's genuinely the more powerful, more employable long-term skill. But at this stage, AWS would mean spending most available time learning VPCs, IAM roles, RDS configuration, and EC2/ECS setup, rather than reinforcing the FastAPI, Postgres, and Alembic concepts already built. It's the difference between learning to drive a car and learning to build one — both are worth learning, but not on the same day.

2.3 Being fair to alternatives

Most platform comparison articles are written by the platform being favorably ranked, including some drawn on directly for this decision — Railway's own blog favors Railway, and Render's own comparison page favors Render, going as far as to cite Railway reliability complaints. Independent sources are more balanced: Railway is generally seen as better for beginners and faster onboarding, while Render is seen as stronger on free-tier database persistence and predictable pricing for steady, always-on workloads. Neither is objectively "correct" — the choice here was made specifically to minimize new platform-specific learning on top of concepts just mastered, not because Railway is universally superior.

Part 3 — The Full Railway Deployment, Step by Step

This section is written as it actually happened — including every real error hit, in order, with the actual reasoning behind each fix. This is intentionally not a cleaned-up "ideal path" — real deployments hit exactly this kind of friction, and knowing how to diagnose it is the actual skill being built.

3.1 Prerequisites

- A GitHub account, since Railway's core workflow is watching a GitHub repo and auto-redeploying on every push.
- A requirements.txt file listing all dependencies, generated with: `pip freeze > requirements.txt` (run as a terminal command, not pasted into the file itself — pasting the command into the file is a real mistake that was caught and fixed here).
- Git installed locally.

3.2 Getting the project onto GitHub

1. Initialize Git in the project folder: `git init`
2. Create a .gitignore excluding `__pycache__/, *.pyc, .env, venv/, *.db`
3. Set Git identity (one-time, per machine): `git config --global user.name / user.email`
4. Stage and commit: `git add .` then `git commit -m "..."`
5. Create an empty repository on GitHub (no README, to avoid conflicts with the existing local history)
6. Connect and push: `git remote add origin <url>`, `git branch -M main`, `git push -u origin main`

Real snag hit: personal files committed by accident

Step 1.pdf, Step 2.docx, and other personal notes were accidentally staged and committed alongside the real project files. Fixed with `git rm --cached <files>` to un-track them without deleting them from disk, then adding their names to .gitignore, then committing and pushing that fix. Lesson: always run `git status` before committing to see exactly what's about to be included.

3.3 Connecting Railway

7. Sign up / log in to Railway using "Login with GitHub", so Railway can see and connect to repos directly.
8. New Project → "Deploy from GitHub repo" → select the project repo.
9. Authorize the Railway GitHub App if prompted.

Railway attempted an automatic build immediately — which failed, as expected at this stage, since no database or start command had been configured yet.

3.4 Real error #1 — wrong entry point (EOFError)

```
File "/app/main.py", line 122, in main
    choice = input("Choose (1-5): ").strip()
EOFError: EOF when reading a line
```

Cause: Railway auto-detected main.py and ran it by convention — but main.py was actually the original CLI version of the project, which calls `input()` expecting a human typing at a keyboard. Railway's server environment has no interactive terminal, so the moment it hit `input()`, there was nothing to read, and Python raised `EOFError`.

Fix: renamed the CLI file out of the way so it could no longer be auto-selected, and explicitly pointed Railway at the real FastAPI app instead.

```
git mv main.py cli_version.py
git commit -m "Rename CLI version out of the way of the FastAPI app"
git push
```

3.5 Real error #2 — no start command detected

```
* No start command detected. Specify a start command:
https://railpack.com/config/file
railpack process exited with an error
```

Cause: with main.py renamed away, Railway's build system (Railpack) no longer had a default file to guess as the entry point, and none had been explicitly configured yet.

Fix: set an explicit Start Command in the Railway dashboard — Service → Settings → Deploy → Start Command:

```
uvicorn api:app --host 0.0.0.0 --port $PORT
```

Part	Why it's needed
api:app	api is the filename (api.py, no extension), app is the FastAPI instance variable inside it (app = FastAPI()).
--host 0.0.0.0	Listens on all network interfaces. Plain localhost would be unreachable from outside the container — Railway's router needs this to route external traffic in.
--port \$PORT	Railway assigns the port dynamically at runtime via this environment variable; a hardcoded port number would likely not match what Railway expects.

3.6 Real error #3 — a Procfile made things worse before they got better

A Procfile (web: uvicorn api:app --host 0.0.0.0 --port \$PORT) was added as an attempted alternative to the dashboard Start Command field, based on the assumption the dashboard setting hadn't saved correctly.

```
* Error reading Procfile as YAML: yaml: unmarshal errors:
  line 1: cannot unmarshal !!str `uvicorn...` into map[string]string
railpack process exited with an error
```

Cause: the Procfile's first line was missing the required web: prefix, so Railway's parser tried to read it as YAML and failed immediately. Railway's own diagnosis message clarified that the Start Command dashboard field had actually been set correctly all along — the Procfile was an unnecessary fix for a problem that didn't exist, and had become the new problem instead.

Extra complication: removing the Procfile, committing, and pushing did not immediately resolve it — Railway kept alternating between asking for a Procfile and rejecting it, consistent with a stale build cache serving an old evaluation rather than the current repo state.

Fix: confirmed via direct GitHub URL check that the Procfile was genuinely gone from the repository, then resolved the stuck cache by deleting the Railway service and recreating it fresh from the same GitHub repo — a fast, common fix for stuck build-cache states.


```
git rm Procfile
git commit -m "Remove Procfile - start command already configured on Railway
service"
git push
```

3.7 Adding Postgres and wiring the connection

10. On the Railway project dashboard: "+ New" → "Database" → "Add PostgreSQL". This provisions a managed Postgres instance as a separate service in the same project.
11. In the app service's Variables tab, added a reference to the database service's connection string:

```
DATABASE_URL = ${Postgres.DATABASE_URL}
```

The `${ServiceName.VariableName}` syntax is Railway's way of referencing another service's variable, so the app always stays correctly wired even if Railway ever rotates the underlying credentials.

3.8 Real error #4 — still connecting to localhost

```
sqlalchemy.exc.OperationalError: (psycopg.OperationalError) connection failed:
connection to server at "127.0.0.1", port 5432 failed: Connection refused
```

Cause: adding the `DATABASE_URL` variable on Railway's side does nothing by itself — the actual Python code in `database.py` was still hardcoded to connect to `localhost:5432`, and had not yet been changed to read the new environment variable at all.

Fix: updated `database.py` to read `DATABASE_URL` from the environment, falling back to the local Postgres connection string when that variable doesn't exist (i.e., when running locally, not on Railway):

```
import os

DATABASE_URL = os.environ.get(
    "DATABASE_URL",
    "postgresql+psycopg://taskadmin:devpassword@localhost:5432/taskdb"
)

if DATABASE_URL.startswith("postgres://"):
    DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql+psycopg://",
1)
elif DATABASE_URL.startswith("postgresql://"):
    DATABASE_URL = DATABASE_URL.replace("postgresql://", "postgresql+psycopg://",
1)

engine = create_engine(DATABASE_URL, pool_size=5, max_overflow=10)
```

This single file now correctly serves both environments: locally, no `DATABASE_URL` variable exists, so it falls back to the local Postgres container; on Railway, the real variable is found and used automatically.

3.9 Real error #5 — wrong Postgres driver

```
ModuleNotFoundError: No module named 'psycopg2'
```

Cause: the first version of the URL-rewrite logic only handled the older `postgres://` prefix. Railway's actual `DATABASE_URL` came through as `postgresql://` (already correct-looking, but still missing the `+psycopg` driver suffix). Since that exact case wasn't handled, the URL passed through unchanged, and SQLAlchemy's default assumption for a driver-less `postgresql://` URL is the older `psycopg2` library — which was never installed in this project (`psycopg v3` was).

Fix: added an explicit `elif` branch to also rewrite the `postgresql://` case, forcing it to always resolve to the installed `psycopg (v3)` driver regardless of which format Railway provided (shown already merged into the code block above).

3.10 Real error #6 — port mismatch on the public domain

```
Application failed to respond
This error appears to be caused by the application.
```

Deploy logs at this point actually showed a fully healthy app:

```
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Cause: when generating the public domain (Settings → Networking → Generate Domain), Railway asks for a Target Port. This had been set to a value that did not match 8080 — the port \$PORT actually resolved to at runtime. Railway's public router was therefore forwarding traffic to the wrong internal port, even though the app itself was running perfectly.

Fix: checked the Deploy Logs for the actual bound port (8080), then updated the domain's Target Port setting to match it exactly.

Lesson worth keeping

When a deployed app shows "failed to respond" but the Deploy Logs show it started cleanly with no errors, the problem is almost always networking/port configuration, not the application code. Always check Deploy Logs first — they tell you definitively whether the app itself is even the problem.

3.11 Final verification

- Opened <https://<app>.up.railway.app/docs> — confirmed the FastAPI Swagger UI loaded with all 4 CRUD routes listed.
- Created a task through the live Swagger UI (POST /tasks).
- Confirmed the task appeared both via GET /tasks on the live URL, and directly inside Railway's Postgres "Data" tab with `SELECT * FROM tasks;` — proving the full chain: public internet → Railway router → FastAPI app → Railway's managed Postgres.

3.12 Known follow-up — Alembic on Railway

At the time of this document, Railway's Postgres has the tasks table (created by `init_db()` on startup) but does not yet have Alembic's tracking table (`alembic_version`). This currently works only because the live schema happens to match the current models. The first time a new column is added locally (e.g. a `user_id` foreign key for the upcoming JWT step), `create_all()` on Railway will silently do nothing, and the live database will fall out of sync with the code — reproducing, in production, the exact gap Alembic was introduced to solve locally.

Action item before the JWT step

Install the Railway CLI, and run `alembic upgrade head` against Railway's live database using Railway's injected environment variables, so its migration history genuinely matches the local one before any new schema changes are made.

Quick Reference — All Commands From This Deployment

Git

```
git init
git status
git add .
git add <file>
git commit -m "message"
git config --global user.name "...
git config --global user.email "...
git rm --cached <file>
git mv <old> <new>
git rm <file>
git remote add origin <url>
git remote -v
git remote remove origin
git branch -M main
git push -u origin main
git push
git log --oneline -5
git commit --allow-empty -m "message"    # force a new commit with no file
changes
```

Railway-specific configuration (not terminal commands — dashboard settings)

Setting	Where / value
Start Command	Service → Settings → Deploy → <code>unicorn api:app --host 0.0.0.0 --port \$PORT</code>
DATABASE_URL	Service → Variables → <code>\${{Postgres.DATABASE_URL}}</code>
Target Port	Service → Settings → Networking → must match the port shown in Deploy Logs (e.g. 8080)

End of Lecture 3. Companion documents: Lecture 2 (PostgreSQL, Docker & Alembic Setup Guide) covers everything before this deployment; the next phase — Alembic on Railway, then JWT authentication — is not yet covered here.